# 《Are LLMs Reliable Rankers? Rank Manipulation via Two-Stage Token Optimization》ACL2026 大白话解读

> 论文核心一句话：**现在大模型经常拿来做排序 / 重排，但是它很容易被简短、看着很正常的文字给“操控”，把某个指定商品强行排到靠前位置。论文提出 RAF 这套两阶段 token 优化方法，专门干这件事；同时暴露出大模型重排的安全漏洞。**

## 一、背景：现在发生了什么

很多搜索、商品推荐系统，不用传统排序算法，改用大模型做**重排（reranker）**。
输入：用户查询 + 一堆候选商品描述；输出：大模型直接给出排好序的商品列表（listwise 范式）。

风险来了：
攻击者可以**只修改目标商品自己的描述文本**，偷偷加一小段文字进去，不需要改系统、不用改大模型权重，就能哄骗大模型，把这个商品排到第一名。而且加进去的那段文字读起来还挺通顺，普通人看不出是攻击，这就叫**排序操纵攻击**。

之前已经有相关攻击方法，但有个两难：

- 要么攻击效果很强，但生成出来的文字乱码、奇怪，一眼就能识别是攻击；
- 要么文字通顺，但是很难把目标商品的排名打上去。

这篇论文的 RAF，目标就是解决这个矛盾：**既能把目标商品排名抬上去，同时生成的附加文本读起来自然流畅，不容易被检测出来**。

> ⚠️ 伦理：论文不是教大家搞恶意攻击；是暴露漏洞，提醒业界要做防御。

## 二、问题定义（通俗版）

1. 有一堆商品，大模型要给它们排序。攻击者挑其中一个**目标商品**。
2. 攻击者不能修改大模型，不能改查询，**只能往目标商品原有描述后面追加一小段攻击文本（RAF Prompt）**。
3. 目标两个：
   ① 效果：让大模型尽量把这个目标商品排第 1；
   ② 隐蔽：追加的文本读起来通顺自然，看不出被篡改。
4. 模型权重全程**固定冻结不动**，只优化输入侧的 token（单词碎片），这叫离散 token 优化。

> 注意：不是微调模型！只是不断挑选、试输入的词，找到能坑大模型的词组合。

## 三、RAF 核心：两阶段 Token 优化（最重要部分）

RAF 是**一个词一个词地生成那一段攻击文本**，每生成一个新词，都要走 Stage1+Stage2 两步。
整体流程图参考论文图 2。

### 两个损失目标（两个要同时考虑的打分）

1. **排序目标损失** $\mathcal{L}_{tar}$
   大白话：衡量 “现在这个词，能不能帮助让目标商品被排到第一名”。损失越小，越有利于抬高排名。

> 实现上看：希望大模型输出排序结果时，优先输出目标商品名字，用交叉熵来算这个损失。

2. **可读性损失** $\mathcal{L}_{read}$
   大白话：衡量 “这个词接在现有句子后面，读起来是否符合人类语言习惯”。
   损失越小，越通顺。计算方式：看大模型本身有多大概率会在这个位置说出这个候选词，概率越高，越自然。

### Stage1：梯度粗筛（快速海选候选词）

词汇表里面几万个词，一个个完整算代价太大。
Stage1 用梯度快速粗筛：把排序损失的梯度 + 可读性损失梯度合并打分，选出分数最高的前 B 个 token，作为候选集合。

> 这一步不做精确计算，只是快速筛掉绝大多数不靠谱单词，缩小搜索范围。

### Stage2：基于动态权重做精确搜索（精细挑选最终词）

拿到 Stage1 筛出来少量候选词之后，对每一个候选词，**实打实完整算出排序损失、可读性损失**。

这里的关键创新：**不是写死一个固定权重把两个损失相加，而是熵驱动动态权重**。

- 熵 $H$：代表大模型对下一个词有多纠结。
- 如果熵低：大模型很确定这里该接什么词 → 多看重**可读性**，优先保证句子通顺；
- 如果熵高：大模型很犹豫不知道接啥 → 多看重**攻击效果（排序提升）**，优先把商品排名打上去。

> 为什么不能固定权重？
> 如果权重固定，要么全程拼命攻击、生成乱码；要么全程追求通顺、攻击失效。动态权重根据上下文自动切换侧重点，才能兼顾两者。

算完组合损失之后，不是直接贪心选最好的那个词；而是带温度 τ 做采样，有一点随机性，避免卡死在局部最优解。选出来的词追加到攻击序列末尾，然后循环，生成下一个 token。

### 收敛停止条件

当最优候选词反复不变，或者组合损失不再下降，就停止，这一个词生成完毕；继续生成下一个词，直到攻击文本达到设定最大长度（论文实验常用 30 个 token）。

## 四、论文发现的 RAF 两种典型失败模式（你做改进任务的绝佳切入点！）

> 这是原论文自己写的局限性，你的改进就可以针对这两个问题之一。

1. **通顺但是攻击无效 Fluent but ineffective**
   当商品原始描述本身上下文很确定（熵很低），动态权重会偏向可读性。RAF 会生成一段读起来非常自然的续写，但完全没有操纵排序的能力。句子很好看，但是商品排名没有提升。

> 根源：熵驱动的动态加权策略带来的固有取舍。

2. **模式崩塌 Mode collapse**
   对很多完全不一样的商品，RAF 最后都生成几乎一模一样的几句万能话术，例如 “不锈钢机身” “用户友好界面”。
   单独看某一条样本，文本通顺，攻击有效；但是整个数据集大量商品都复用相同片段。防御方只要简单检查重复短语，就能批量识别攻击。

> 原 RAF 只看单个样本的损失，**完全没有考虑跨样本之间的多样性惩罚**。这是一个很明显的局限。

## 五、实验怎么测？三个评价指标

1. **平均排名 average rank**：越低越好，代表目标商品排得越靠前。基线无攻击的时候平均是 5.5。
2. **困惑度 Perplexity (PPL)**：越低越好，代表追加后的商品文本流畅度高。
3. **坏词率 bad-word ratio**：统计有没有出现 “top、first、must recommend” 这类直白推销词；越低越隐蔽。

对比基线：

- STS：老的梯度攻击，攻击强但是文字大量乱码；
- StealthRank (SRP)：另一个隐蔽攻击，先优化连续向量，再转真实 token，转换一步会带来性能崩坏。

实验结果：RAF 对比两者，**平均排名更低（攻击更强），同时 PPL 更低（文本更通顺），坏词率差不多**，兼顾攻击效果和隐蔽性。另外 RAF 生成的攻击 prompt 还具备一定跨模型迁移能力：在 A 模型优化出来的攻击文本，丢给另一个没见过的模型，依然还能起一部分作用。

还做了消融实验：

- 如果删掉可读性损失：排名效果反而变差，很难收敛，大量乱码；
- 如果删掉排序损失：文本极度通顺，但完全无法操纵排名。
  证明两个目标缺一不可。

## 六、和你导师布置任务关联总结

1. RAF 本质：**冻结大模型权重，离散 token 两阶段优化，同时优化【排序攻击效果 + 文本可读性】，熵动态权重平衡二者**。
2. 原生方法的硬伤（你可以改进的方向）

- 问题①：熵驱动动态权重会出现 “太通顺但是攻击失效” 的情况；
- 问题②：模式崩塌，多个样本生成高度雷同攻击片段，容易被 n-gram 重复检测防御；
- 问题③：Stage1 梯度粗筛和 Stage2 精确评估存在 gap，粗筛阶段漏过一些潜在好 token。

3. 不要混淆：
   - ✅ RAF：直接在真实 token 空间优化；
   - ❌ StealthRank：先优化虚拟连续 embedding，再映射为真实 token，存在映射损失。

如果你需要，我下一步可以帮你把：损失函数、算法伪代码 Algorithm1 也翻译成通俗版本，或者结合代码仓库给你讲代码对应哪一部分。